



Artificial Intelligence Algorithms for Electronic Component Recognition

Author: Giovanni Cadau

Master Degree Thesis in Data Science and Engineering

Advisor: Prof. Daniele Apiletti

Company advisors: Paolo Chiartano, Ing. Francesco Grassino

A.Y. 2023/2024

Presentation outline

- Context and Rationale
 - Electronic Testing
 - Automatic Component Recognition
- Model Design and Architecture
 - Data Collection
 - Data Preprocessing
 - Data Augmentation
 - Model Architecture
- Model Training and Evaluation
 - Model Training
 - Model Evaluation

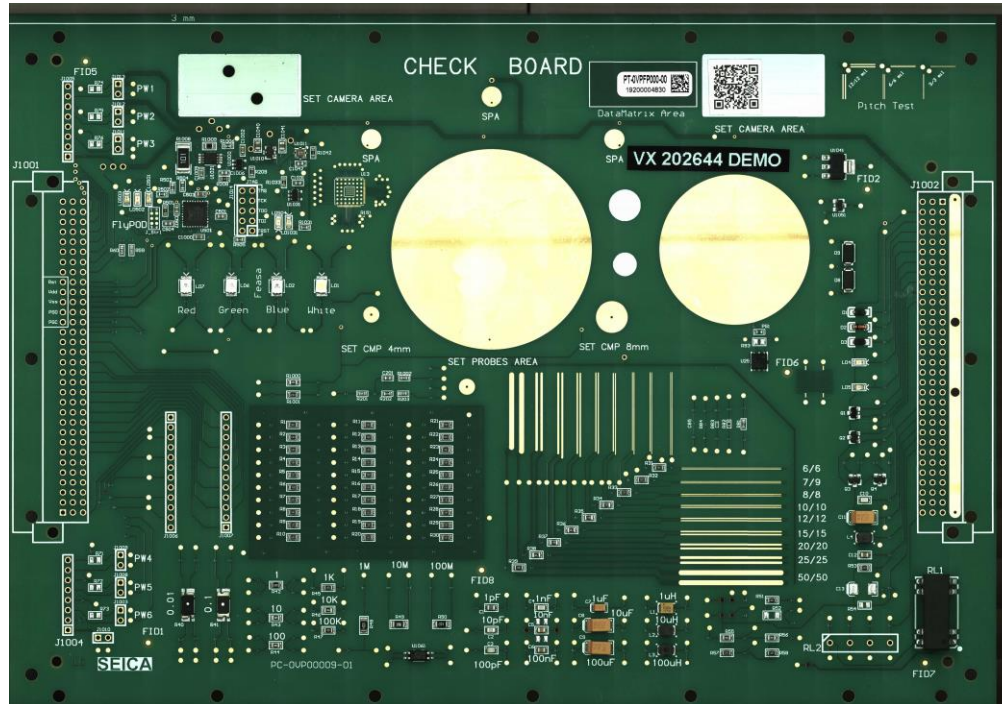
Presentation outline

- (Adaptive) Domain Randomization
 - Domain Randomization
 - Adaptive Domain Randomization
 - Model Robustness Evaluation
- Model Application
 - Image Classification
 - Object Detection
- Performance Optimization in Industrial Systems
 - Key Factors
 - Main Approaches
 - Optimization techniques Effectiveness Evaluation



Seica S.p.A.'s [Flying Probe](#), used for Electronic Testing of PCBs

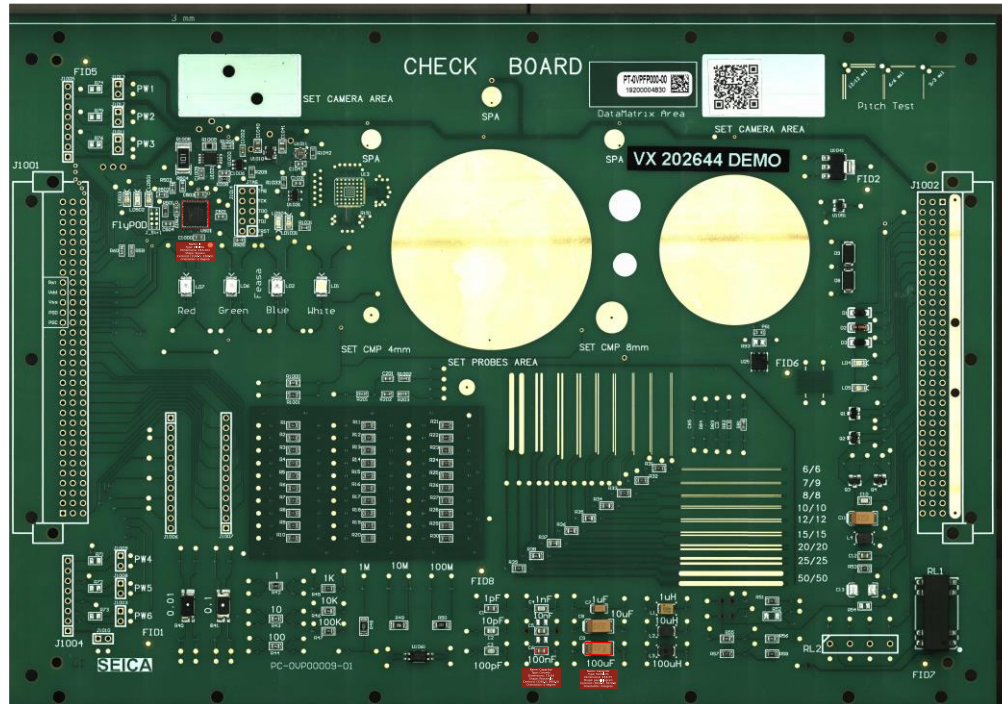
Test of Printed Circuit Boards (PCBs)



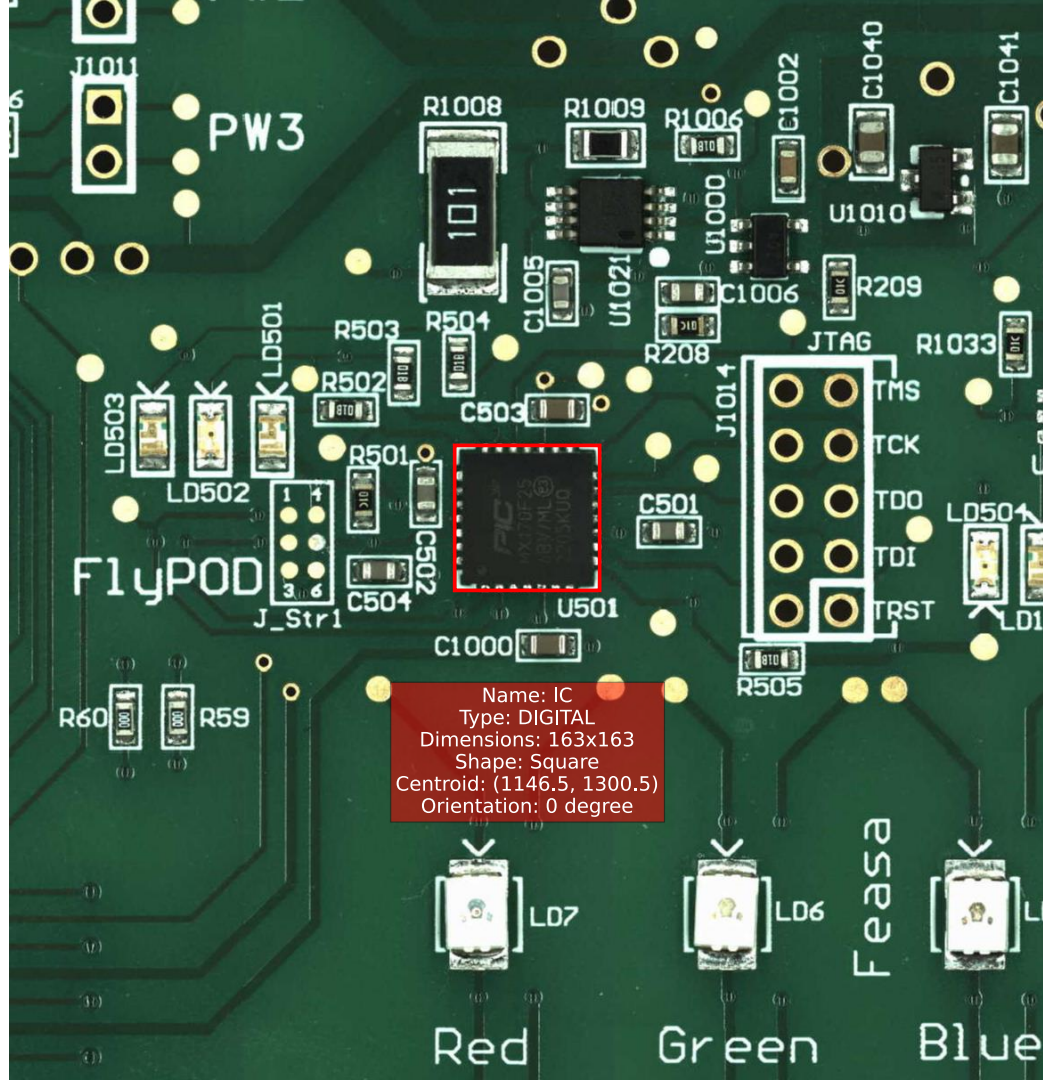
A Printed Circuit Board

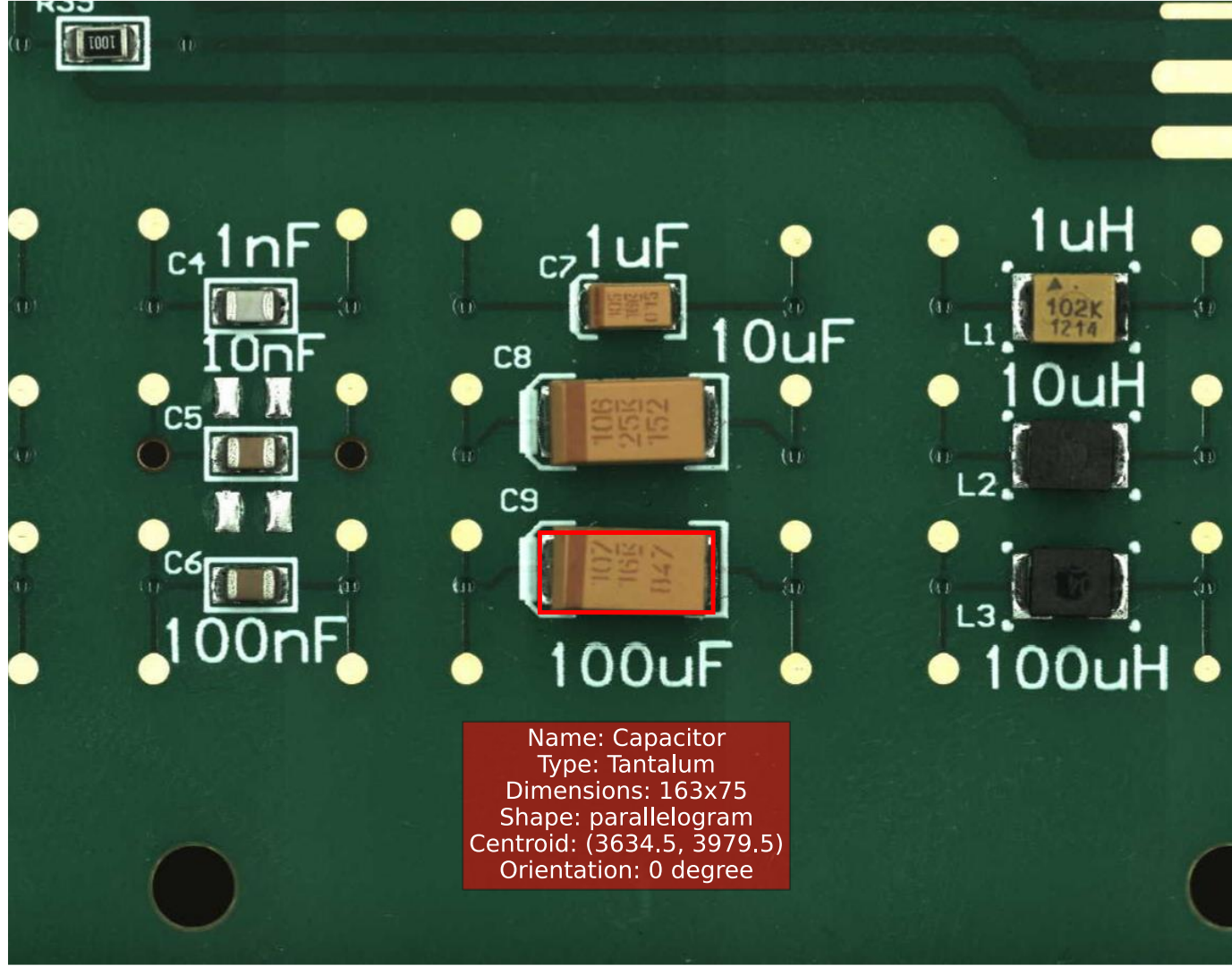
Test of Printed Circuit Boards (PCBs)

- Crucial role: Recognition of specific Electronic Components



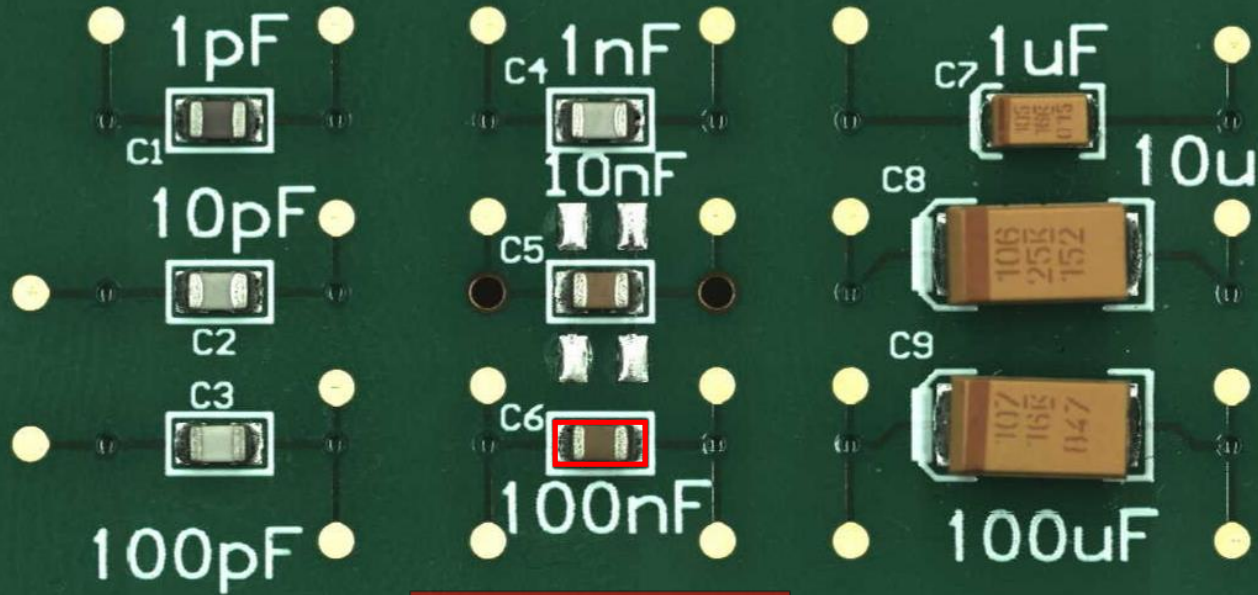
Zooming in...





Name: Capacitor
Type: Tantalum
Dimensions: 163x75
Shape: parallelogram
Centroid: (3634.5, 3979.5)
Orientation: 0 degree

FID8



Name: Capacitor
Type: Ceramic
Dimensions: 73x34
Shape: Rectangle
Centroid: (3281.5, 3991.0)
Orientation: 0 degree

Test of Printed Circuit Boards (PCBs)

- Crucial role: Recognition of specific Electronic Components

It requires:

- an extensive amount of meticulous manual labour
- the presence of a domain expert

Test of Printed Circuit Boards (PCBs)

- Crucial role: Recognition of specific Electronic Components

It requires:

- an extensive amount of meticulous manual labour
- the presence of a domain expert

Limitations:

- Time-consuming process
- Error-prone task
- Domain expert requirement

Test of Printed Circuit Boards (PCBs)

- Crucial role: Recognition of specific Electronic Components

It requires:

- an extensive amount of meticulous manual labour
- the presence of a domain expert

Limitations:

- Time-consuming process
- Error-prone task
- Domain expert requirement

Automation through AI models, potential benefits:

- Increased efficiency
- Improved accuracy
- Scalability

Automatic Component Recognition

Context and
Rationale

Challenges

Automatic Component Recognition

Context and
Rationale

Challenges

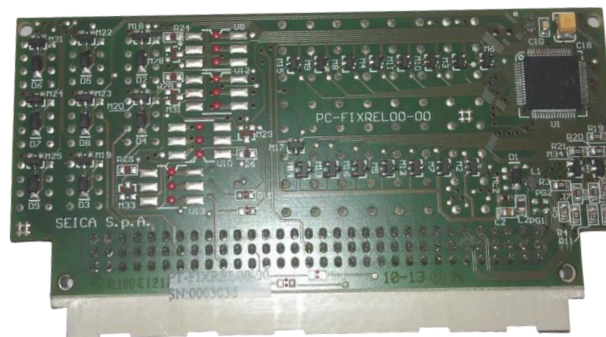
- Different PCB designs

Automatic Component Recognition

Context and
Rationale

Challenges

- Different PCB designs



Challenges

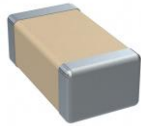
- Different PCB Designs
- Component Variability

Automatic Component Recognition

Context and
Rationale

Challenges

- Different PCB Designs
- Component Variability

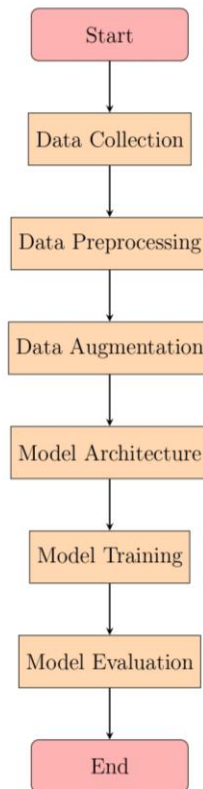


Challenges

- Different PCB Designs
- Component Variability
- [Environmental Factors](#)

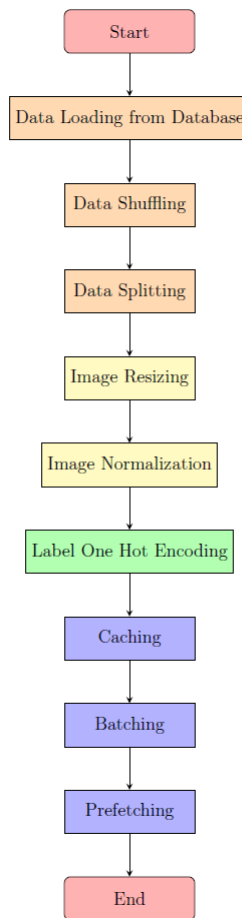
Challenges

- Different PCB Designs
- Component Variability
- Environmental Factors
- [Manufacturing Variations](#)



Data sources

- High quality photos of PCBs used in Seica's systems
 - partially manual and partially automated through software
- Images stored in Seica's database
- Images annotated with a specific label describing the component



- Brightness adjustment
- Contrast adjustment
- Horizontal flip
- Vertical flip
- Hue adjustment
- Jpeg quality adjustment
- Saturation adjustment

Different model architectures proposed:

- Support Vector Machine

Feature Engineering:

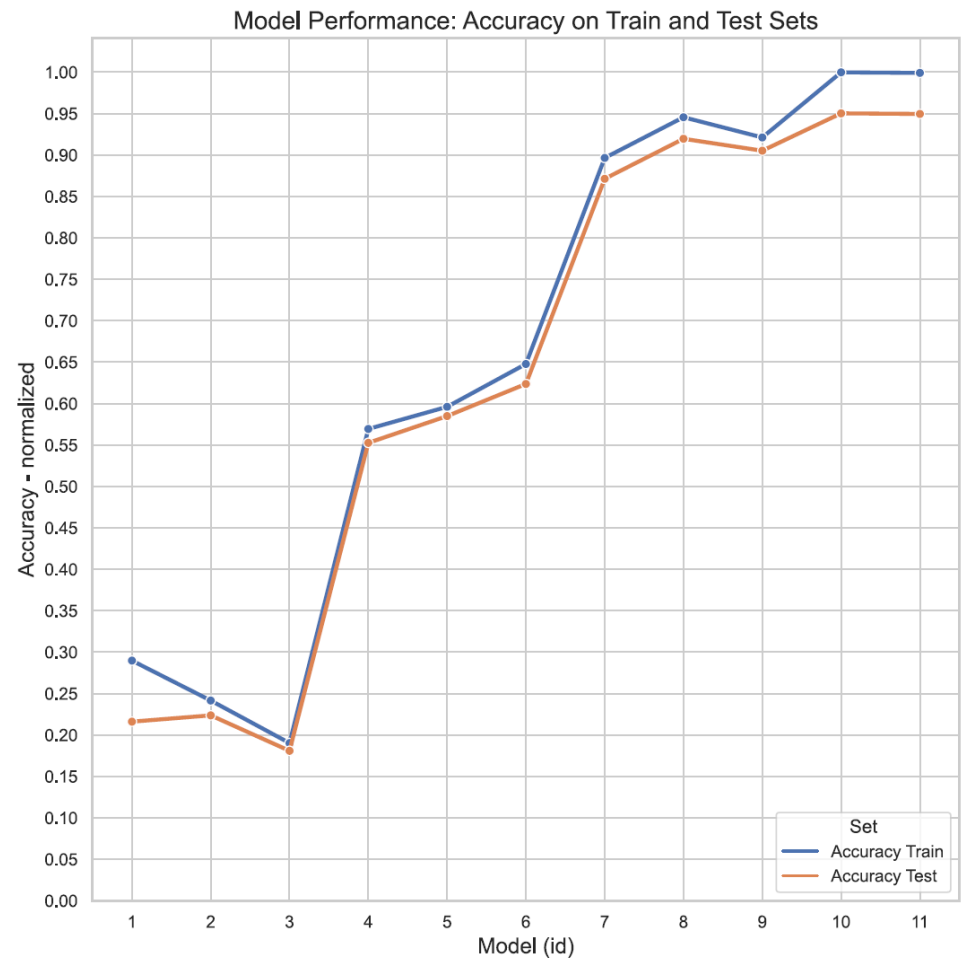
- Histogram of Oriented Gradients
- Principal Component Analysis
- Local Binary Patterns

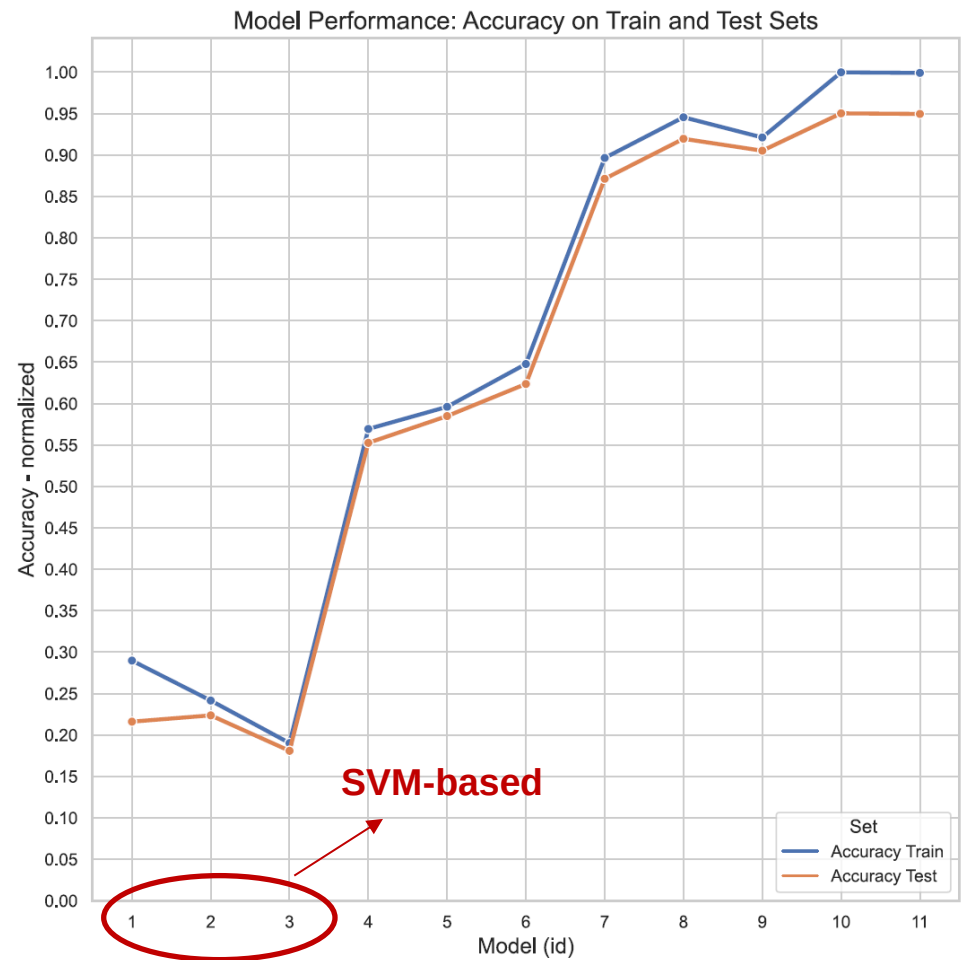
- Convolutional Neural Network
- Residual Neural Network

(See Appendix for details on the 11 Models)

- Cross-validation: 5-fold cross-validation
- Neural Network training parameters:
 - Epochs: 1000
 - Learning rate: 0.001
 - Optimizer: Gradient-based optimization algorithms
(several tested, Adam used for evaluation phase)
 - Loss function: several tested, Categorical Crossentropy used for evaluation phase

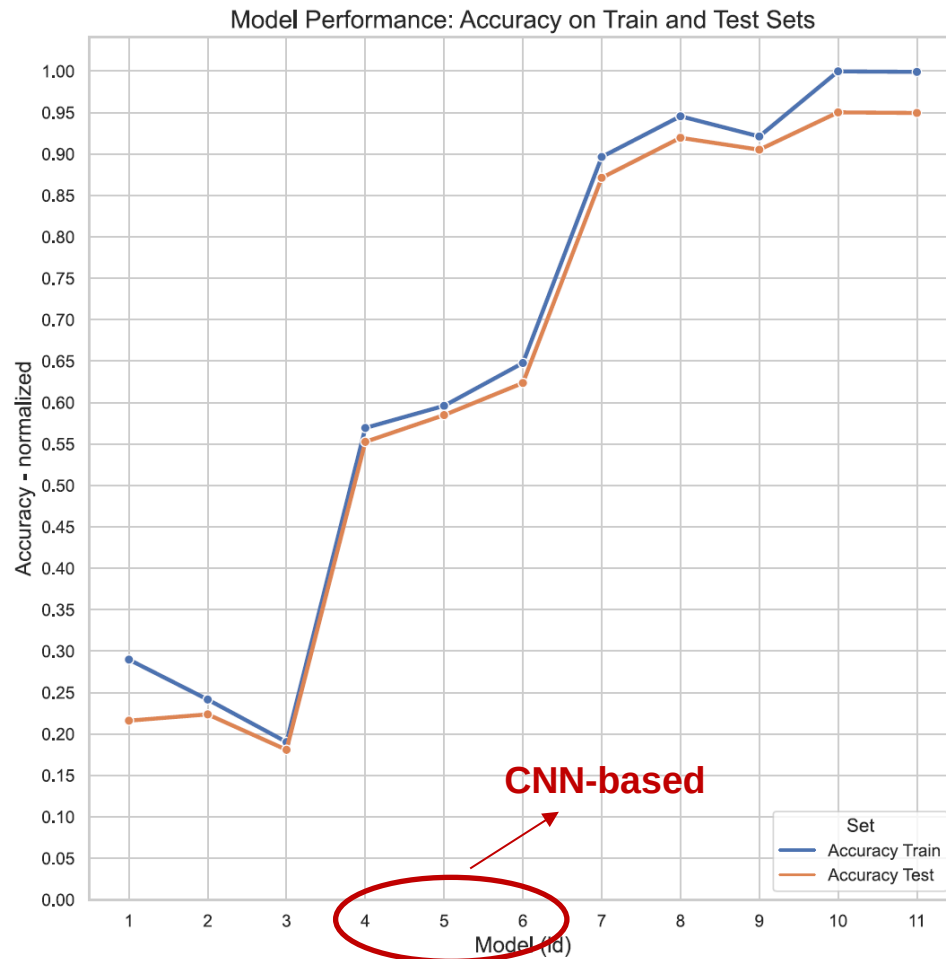
(See [Appendix](#) for details on the 11 Models evaluated)

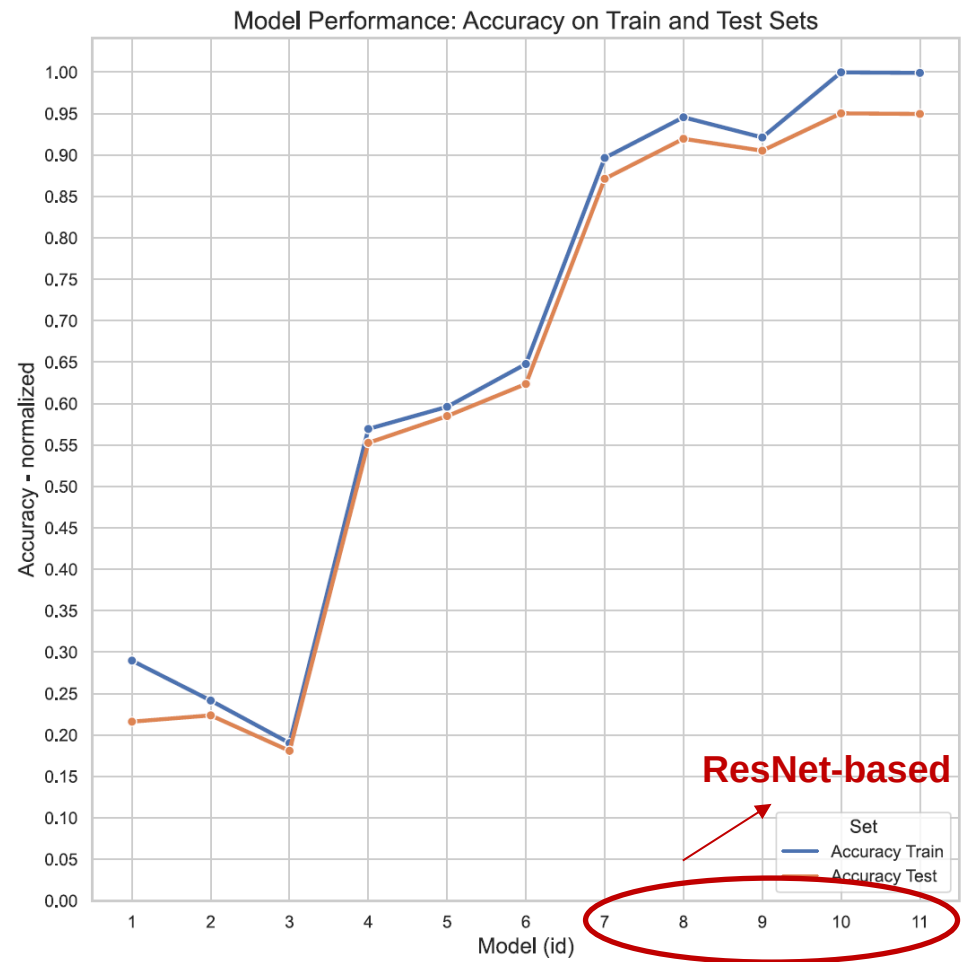




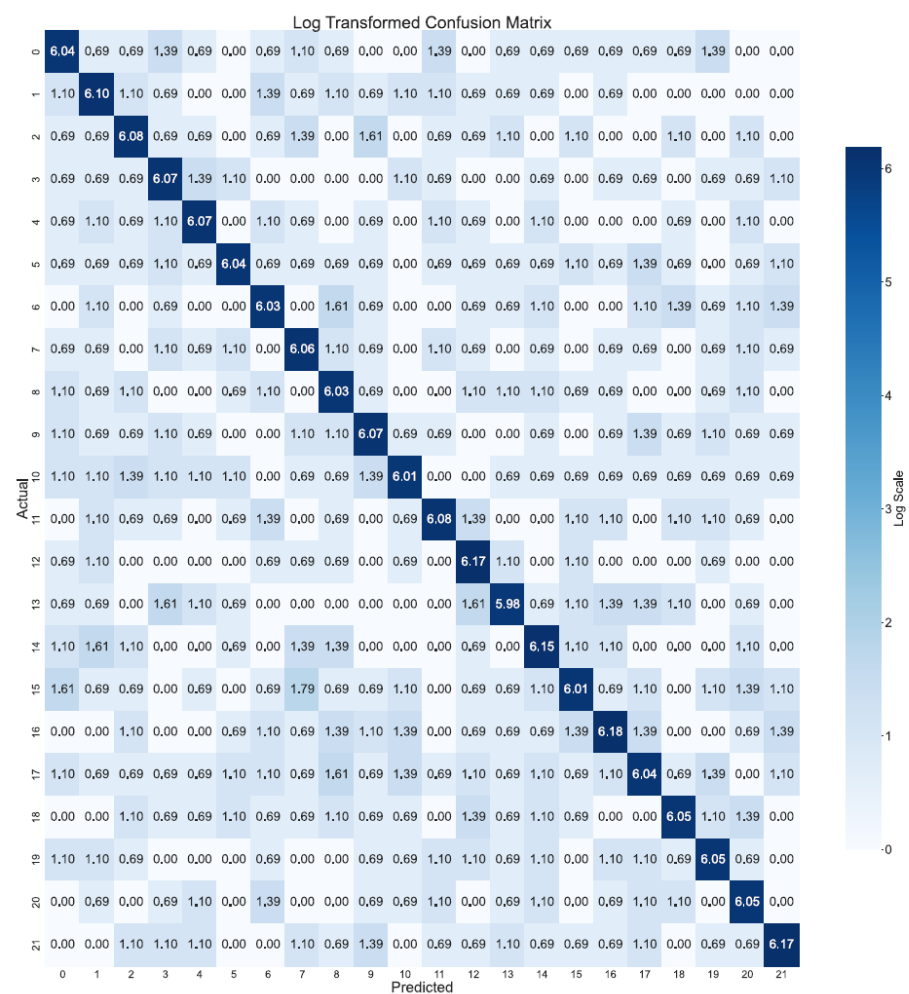
Model Evaluation

Model Training and Evaluation





Model ID	Accuracy (%)		Precision (%)		Recall (%)		F1-Score (%)	
	Train	Test	Train	Test	Train	Test	Train	Test
1	28.98	21.61	29.00	21.61	29.00	21.64	28.97	21.61
2	24.16	22.37	24.16	22.39	24.19	22.40	24.16	22.38
3	19.03	18.09	19.04	18.09	19.05	18.11	19.03	18.08
4	56.94	55.26	56.98	55.25	56.96	55.28	56.92	55.23
5	59.61	58.49	59.59	58.48	59.63	58.51	59.59	58.46
6	64.78	62.37	64.79	62.36	64.81	62.40	64.77	62.36
7	89.65	87.13	89.62	87.11	89.64	87.10	89.62	87.09
8	94.56	91.96	94.56	91.95	94.54	91.95	94.54	91.94
9	92.12	90.52	92.12	90.51	92.11	90.51	92.11	90.50
10	99.98	95.03	99.98	95.01	99.98	95.03	99.98	95.01
11	99.91	94.96	99.91	94.94	99.91	94.94	99.91	94.94



- Idea: use Data Augmentation techniques to enhance the robustness and generalization capabilities of developed models: help model generalize to new, unseen environments
- Expose the model to artificially generated variations during training

- Idea: use Data Augmentation techniques to enhance the robustness and generalization capabilities of developed models: help model generalize to new, unseen environments
- Expose the model to artificially generated variations during training

Specific domain represented by a set of parameters (image properties):

- Brightness
- Contrast
- Horizontally flip
- Vertically flip
- Hue
- Jpeg quality
- Saturation

- How Parameters are Randomized

Parameter Randomization Algorithm

Require: Parameters set $\mathbf{P}$, the list of parameters to be randomized

Require: Probability vector $\mathbf{p}$, where $\mathbf{p}_i$ is the probability that the i -th parameter is randomized

Require: Number of epochs E

Require: Batch size B

```
1: for each epoch  $e$  from 1 to  $E$  do
2:   for each batch  $b$  from 1 to  $B$  do
3:     for each parameter  $i$  in  $\mathbf{P}$  do
4:       if  $\text{random.rand}() > \mathbf{p}_i$  then
5:         Sample a value for the parameter  $i$  from the parameter distribution  $\mathcal{D}_i$ 
6:         Generate an augmented image to be used for training
7:       end if
8:     end for
9:   end for
10: end for
```

- Distributions used

Uniform distributions

- $\mathcal{U}(a_{\text{brightness}}, b_{\text{brightness}})$
- $\mathcal{U}(a_{\text{contrast}}, b_{\text{contrast}})$
- $\mathcal{U}(a_{\text{horizontal flip}}, b_{\text{horizontal flip}})$
- $\mathcal{U}(a_{\text{vertical flip}}, b_{\text{vertical flip}})$
- $\mathcal{U}(a_{\text{hue}}, b_{\text{hue}})$
- $\mathcal{U}(a_{\text{jpeg quality}}, b_{\text{jpeg quality}})$
- $\mathcal{U}(a_{\text{saturation}}, b_{\text{saturation}})$

with $a_i, b_i, i \in \mathbf{P}$ representing the lower and upper bounds of the uniform distribution for each parameter, respectively.

- Distributions used

Triangular distributions

- $\mathcal{T}(a_{\text{brightness}}, m_{\text{brightness}}, b_{\text{brightness}})$
- $\mathcal{T}(a_{\text{contrast}}, m_{\text{contrast}}, b_{\text{contrast}})$
- $\mathcal{T}(a_{\text{horizontal flip}}, m_{\text{horizontal flip}}, b_{\text{horizontal flip}})$
- $\mathcal{T}(a_{\text{vertical flip}}, m_{\text{vertical flip}}, b_{\text{vertical flip}})$
- $\mathcal{T}(a_{\text{hue}}, m_{\text{hue}}, b_{\text{hue}})$
- $\mathcal{T}(a_{\text{jpeg quality}}, m_{\text{jpeg quality}}, b_{\text{jpeg quality}})$
- $\mathcal{T}(a_{\text{saturation}}, m_{\text{saturation}}, b_{\text{saturation}})$

with a_i , b_i , m_i , $i \in \mathbf{P}$ representing the lower bound, mode, and upper bound of the triangular distribution for each parameter, respectively.

- Distributions used

Univariate normal distributions

- $\mathcal{N}(\mu_{\text{brightness}}, \sigma_{\text{brightness}}^2)$
- $\mathcal{N}(\mu_{\text{contrast}}, \sigma_{\text{contrast}}^2)$
- $\mathcal{N}(\mu_{\text{horizontal flip}}, \sigma_{\text{horizontal flip}}^2)$
- $\mathcal{N}(\mu_{\text{vertical flip}}, \sigma_{\text{vertical flip}}^2)$
- $\mathcal{N}(\mu_{\text{hue}}, \sigma_{\text{hue}}^2)$
- $\mathcal{N}(\mu_{\text{jpeg quality}}, \sigma_{\text{jpeg quality}}^2)$
- $\mathcal{N}(\mu_{\text{saturation}}, \sigma_{\text{saturation}}^2)$

with $\mu_i, \sigma_i^2, i \in \mathbf{P}$ representing the mean and variance of the normal distribution for each parameter, respectively.

- Distributions used

Multivariate normal distribution

$$\mathcal{N}_7 \left(\begin{bmatrix} \mu_{\text{brightness}} \\ \mu_{\text{contrast}} \\ \mu_{\text{horizontal flip}} \\ \mu_{\text{vertical flip}} \\ \mu_{\text{hue}} \\ \mu_{\text{jpeg quality}} \\ \mu_{\text{saturation}} \end{bmatrix}, \begin{bmatrix} \sigma_{\text{brightness}}^2 & \sigma_{\text{brightness,contrast}} & \cdots & \sigma_{\text{brightness,saturation}} \\ \sigma_{\text{contrast,brightness}} & \sigma_{\text{contrast}}^2 & \cdots & \sigma_{\text{contrast,saturation}} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{\text{saturation,brightness}} & \sigma_{\text{saturation,contrast}} & \cdots & \sigma_{\text{saturation}}^2 \end{bmatrix} \right)$$

μ_i : mean, σ_i^2 : variance, and σ_{ij} : covariance

with $i, j \in \mathbf{P}, i \neq j$

- Challenge in Domain Randomization Approach: find suitable values for the distribution coefficients: $a_i, b_i, m_i, \mu_i, \sigma_i^2, \sigma_{ij}$ with $i \in \mathbf{P}$
- Use of optimization algorithms: systematically find the set of coefficients that provide the best performance for the model
 - Parallel branch in the Neural Network added, using a Gradient-free optimizer

- How Parameters are Randomized

Parameter Randomization and Optimization Algorithm

Require: Parameters set $\mathbf{P}$, the list of parameters to be randomized

Require: Probability vector $\mathbf{p}$, where $\mathbf{p}_i$ is the probability that the i -th parameter is randomized

Require: Number of epochs E

Require: Batch size B

Require: Distribution $\mathcal{D}_i(\mathbf{c}_i)$ for each parameter i , where $\mathbf{c}_i$ are the coefficients to be optimized

```
1: for each epoch  $e$  from 1 to  $E$  do
2:   for each batch  $b$  from 1 to  $B$  do
3:     for each parameter  $i$  in  $\mathbf{P}$  do
4:       if  $\text{random.rand}() > \mathbf{p}_i$  then
5:         Sample a value for the parameter  $i$  from the optimized distribution  $\mathcal{D}_i(\mathbf{c}_i)$ 
6:         Generate an augmented image to be used for training
7:         Forward propagation in the Neural Network
8:         Evaluate Loss  $\mathcal{L}$ 
9:         Optimize coefficients  $\mathbf{c}_i$  of the distribution  $\mathcal{D}_i(\mathbf{c}_i)$ , based on the value of  $\mathcal{L}$ 
10:      end if
11:    end for
12:  end for
13: end for
```

- Model Robustness evaluation

Further evaluation performed on Model 10

➤ *New dataset* introduced:

images significantly different from those stored in the *Old dataset*

Model	Acc Test	Acc Train
Baseline (no rand.)	82.46%	99.98%
Rand., $\mathcal{U}$ dist	89.91%	91.45%
Rand., $\mathcal{T}$ dist	90.31%	92.65%
Rand., Univ $\mathcal{N}$ dist	89.25%	92.84%
Rand., Multiv $\mathcal{N}$ dist	87.88%	89.13%
Rand., $\mathcal{U}$ dist + opt	93.24%	94.22%
Rand., $\mathcal{T}$ dist + opt	91.39%	93.57%
Rand., Univ $\mathcal{N}$ dist + opt	91.85%	92.99%
Rand., Multiv $\mathcal{N}$ dist + opt	99.88%	99.93%

- Model Robustness evaluation

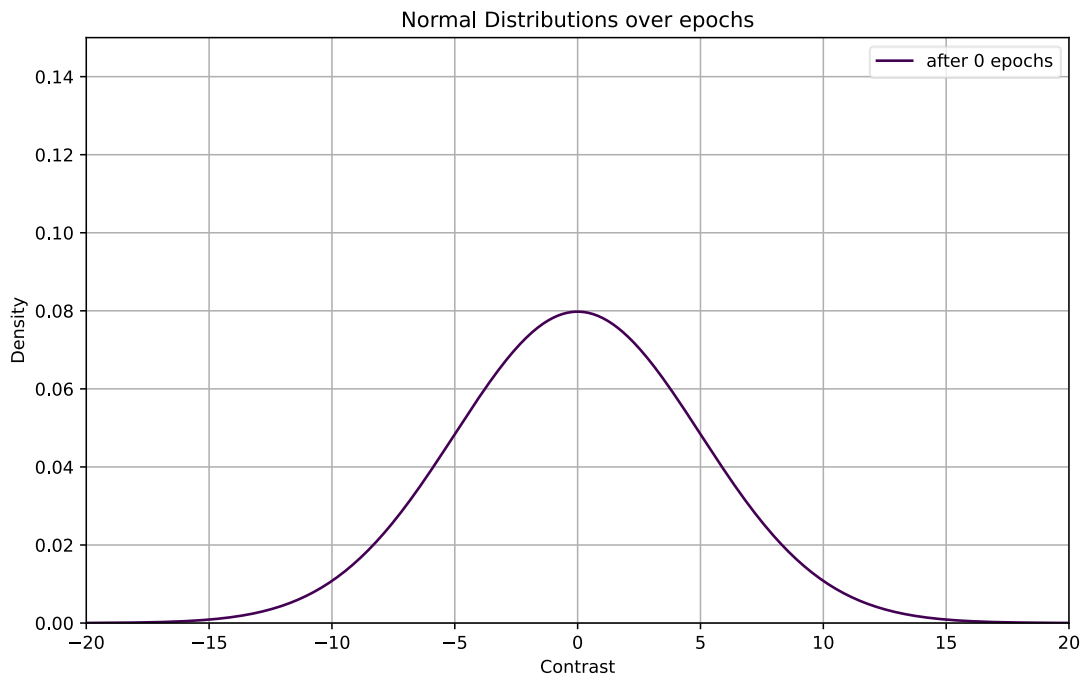
Further evaluation performed on Model 10

- *New dataset* introduced:
images significantly different from those stored in the *Old dataset*

(See [Appendix](#) for the optimal distribution found)

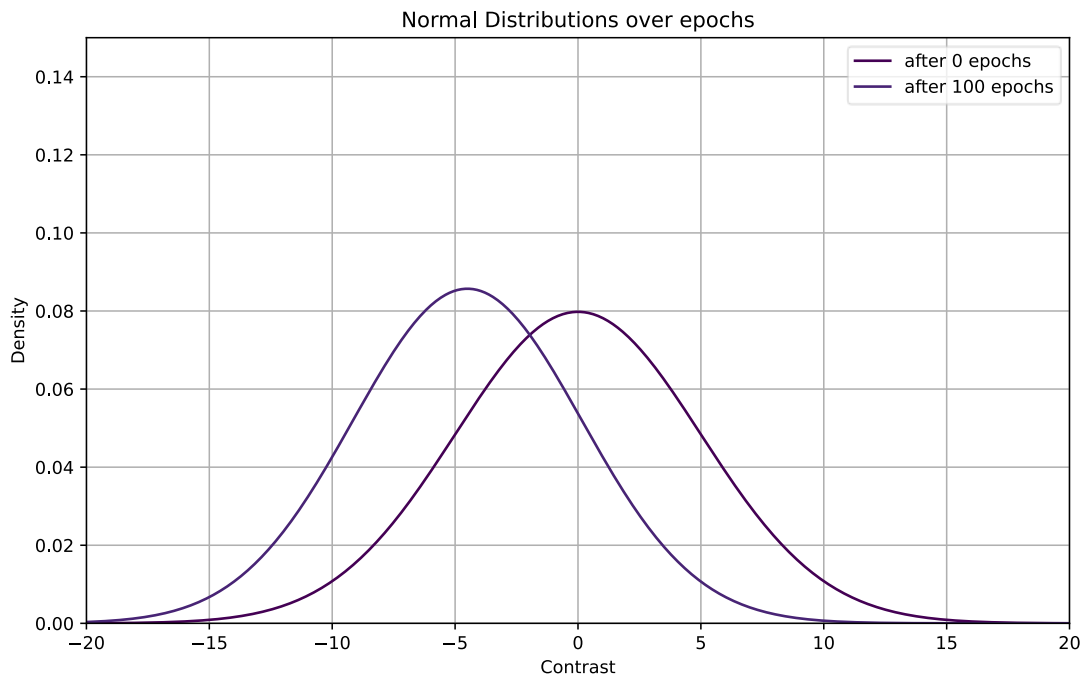
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



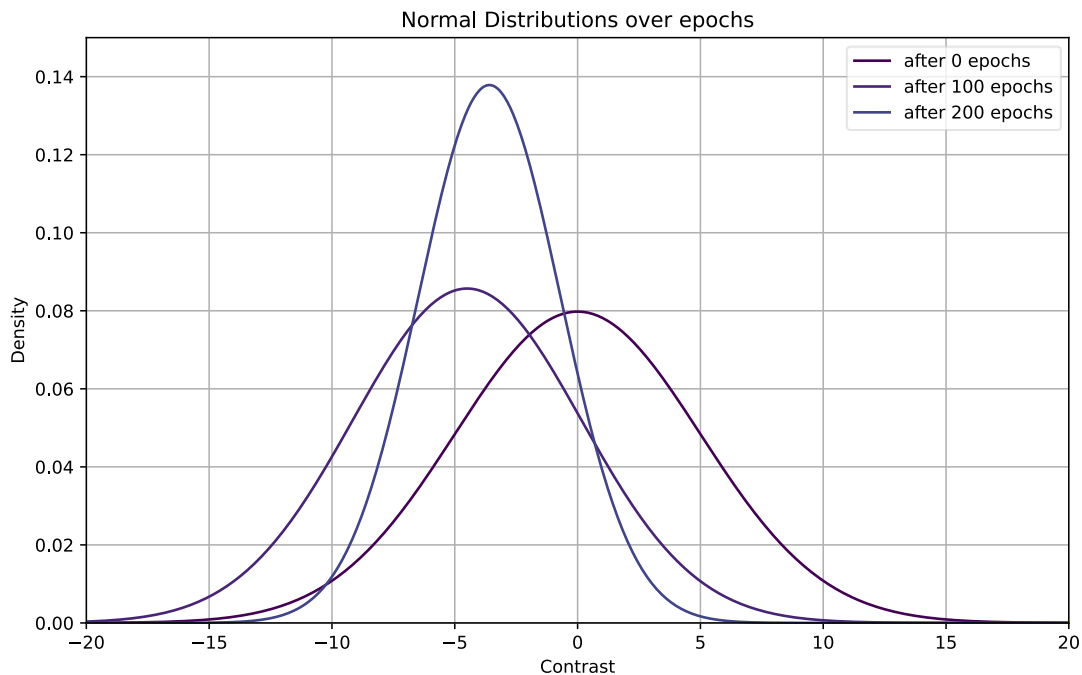
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



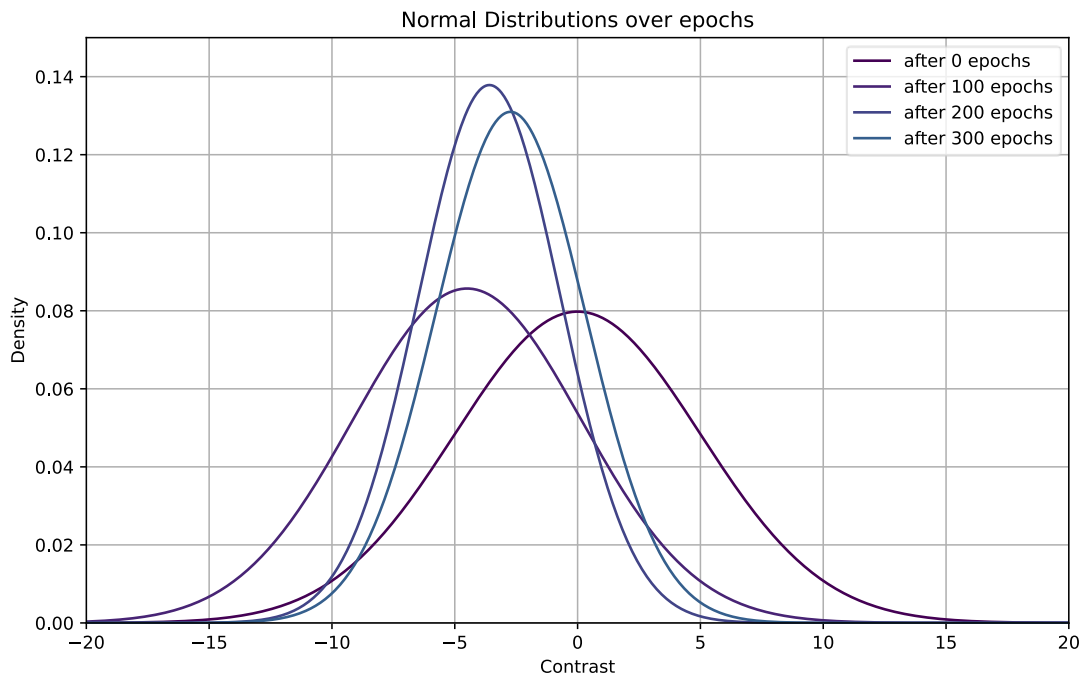
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



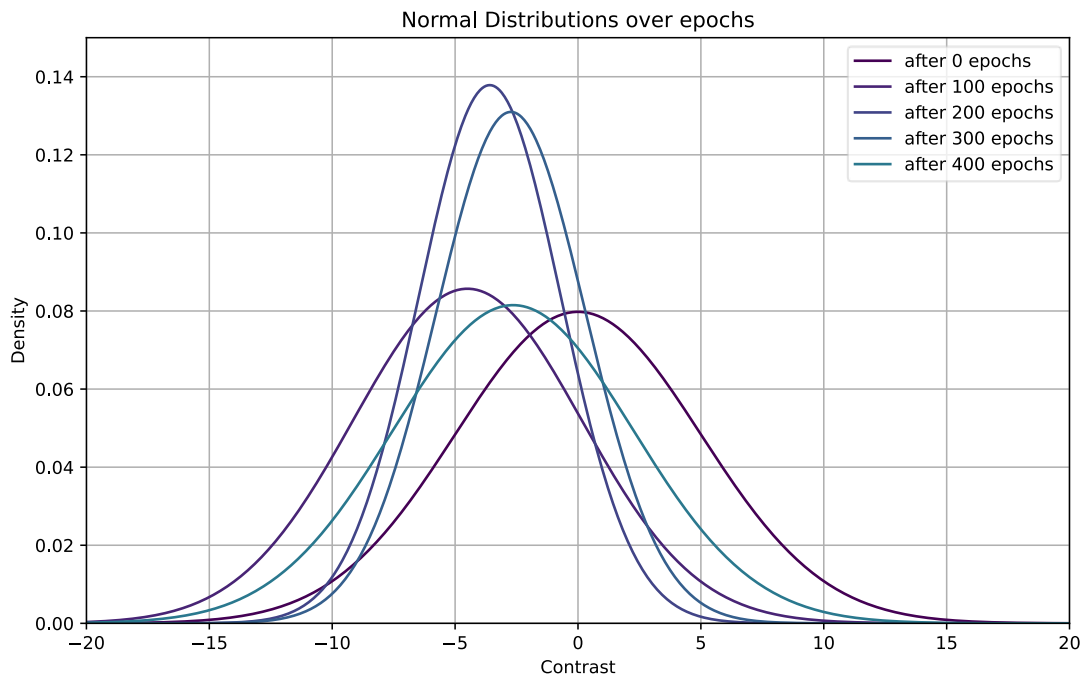
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



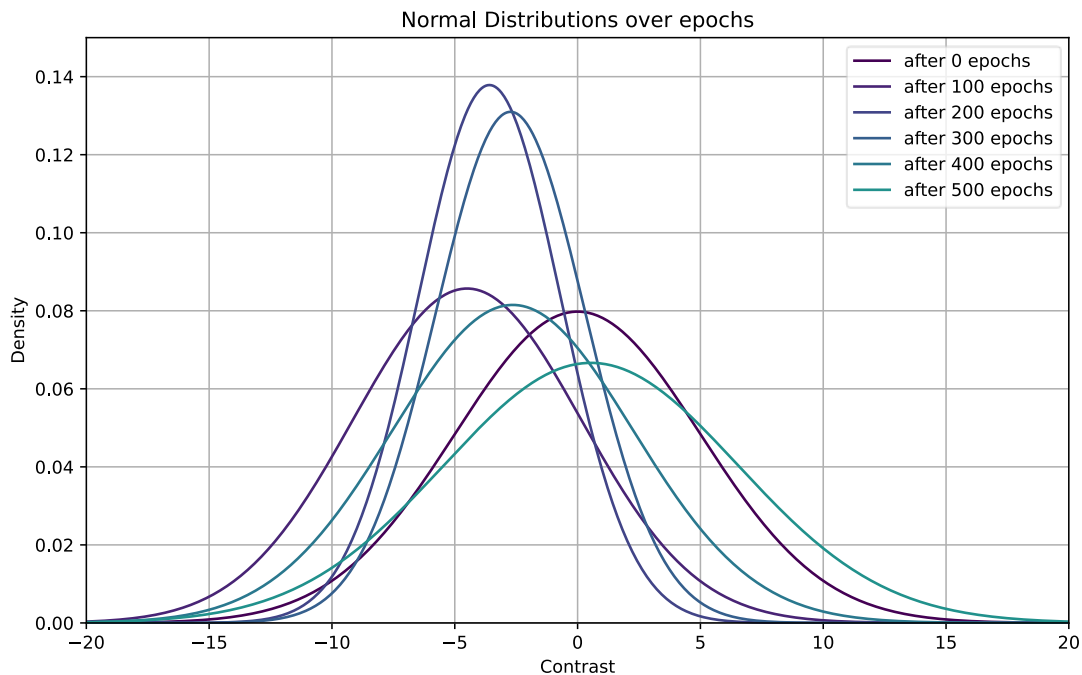
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



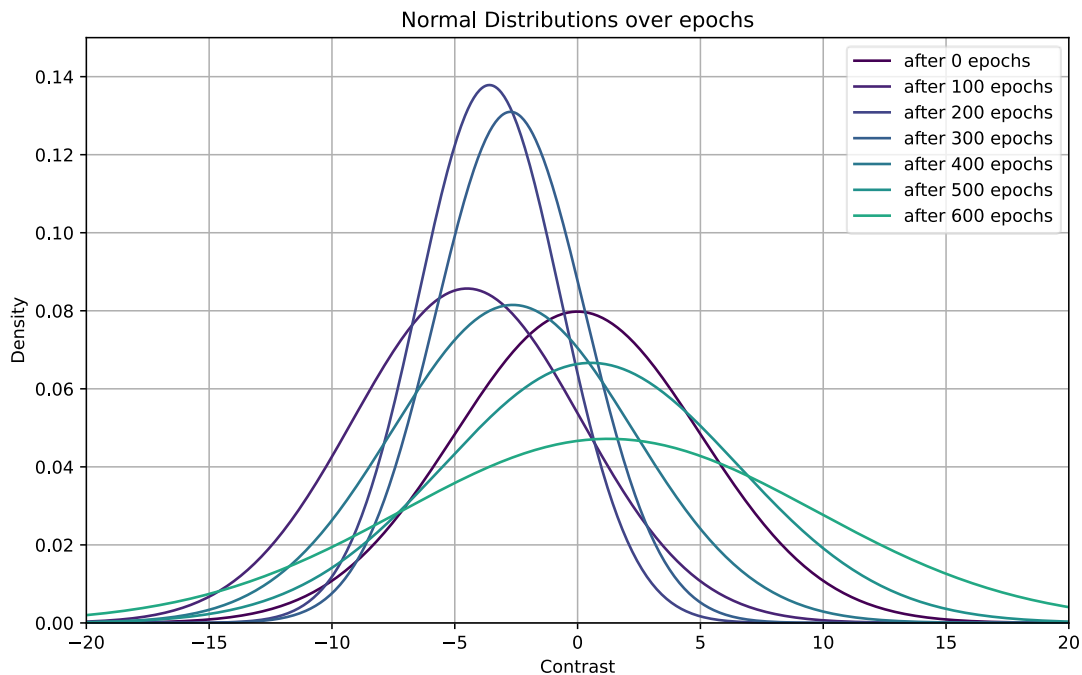
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



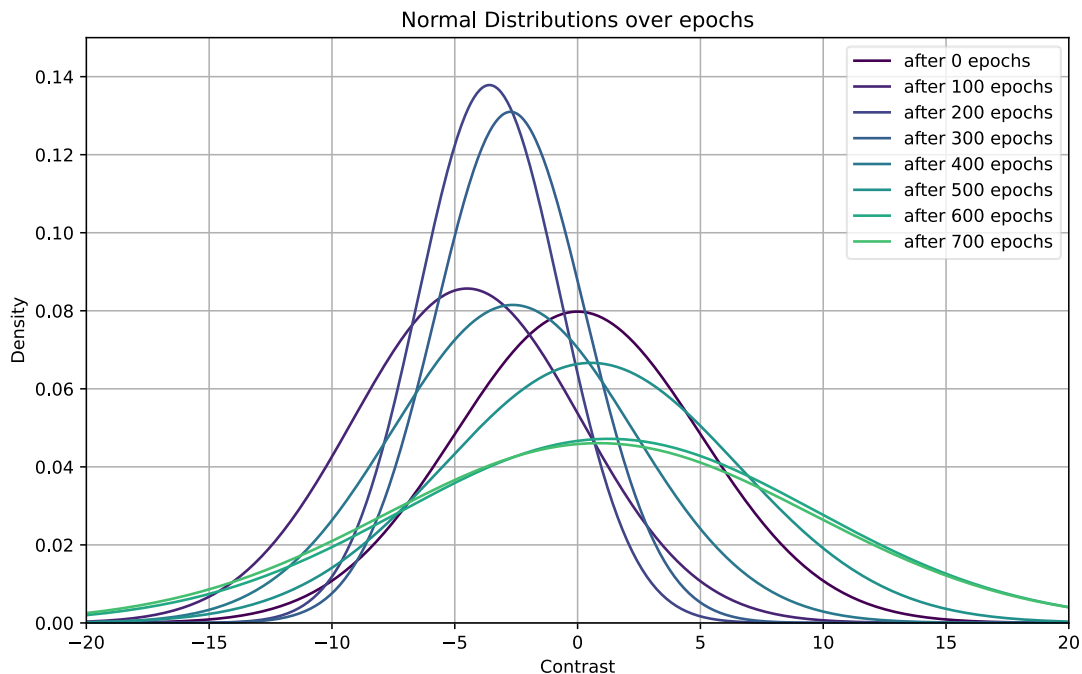
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



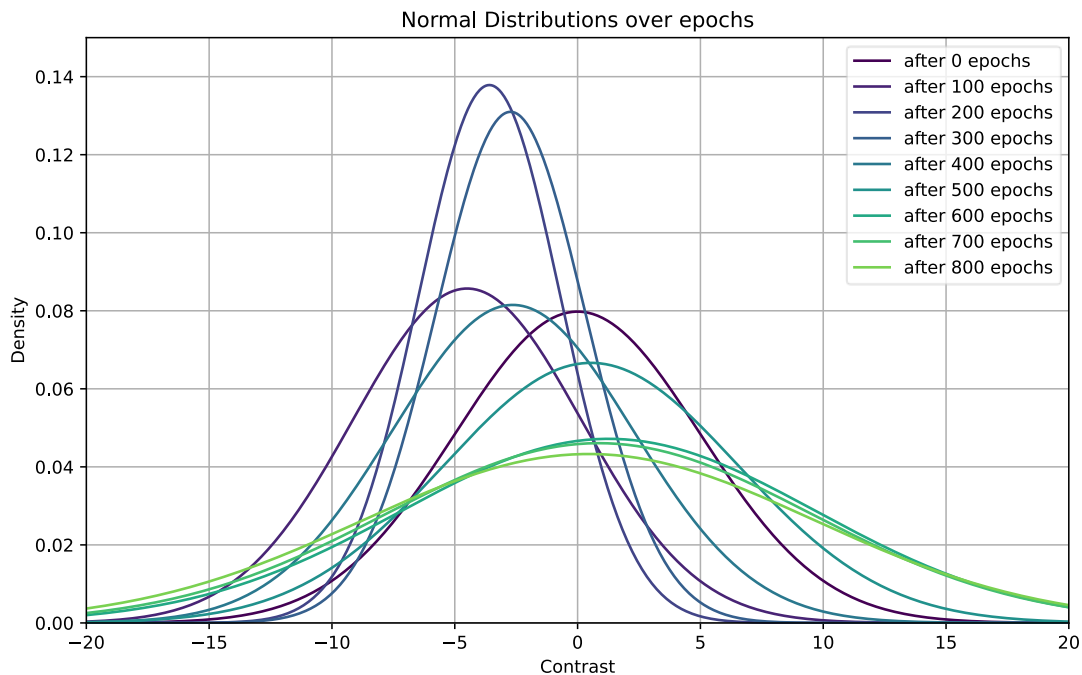
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



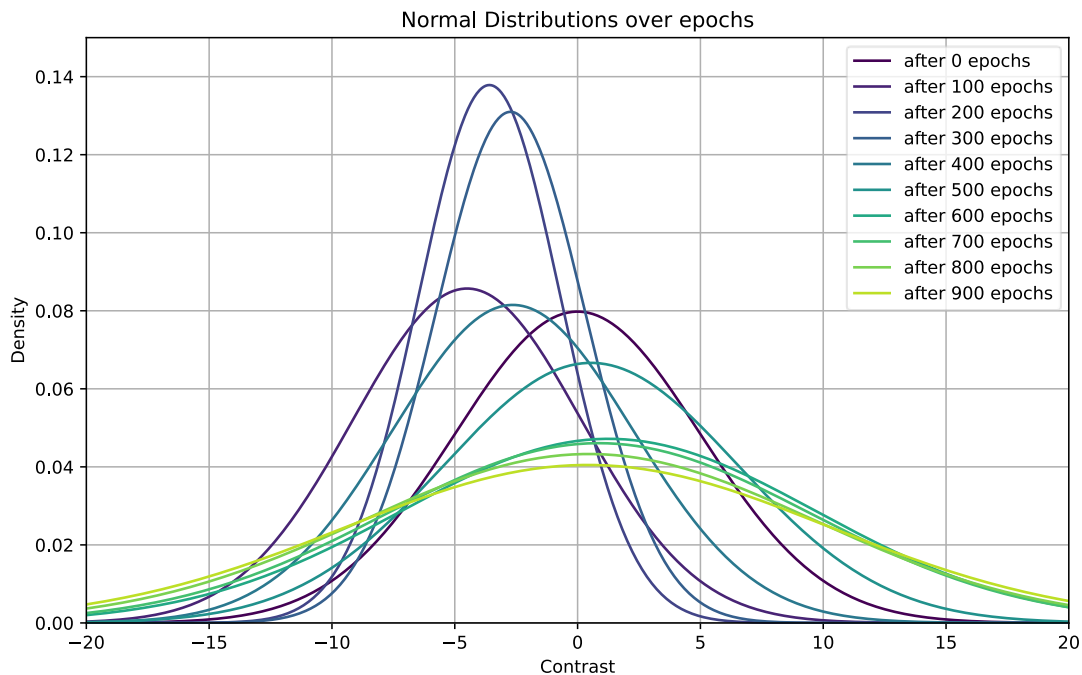
- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



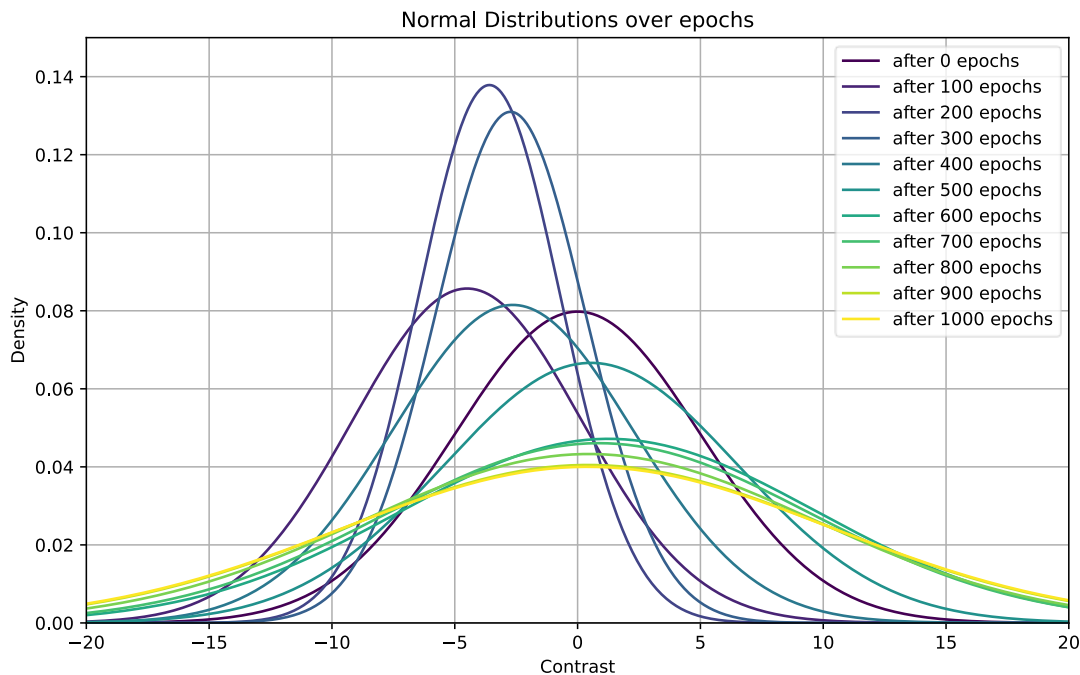
- Model Robustness evaluation

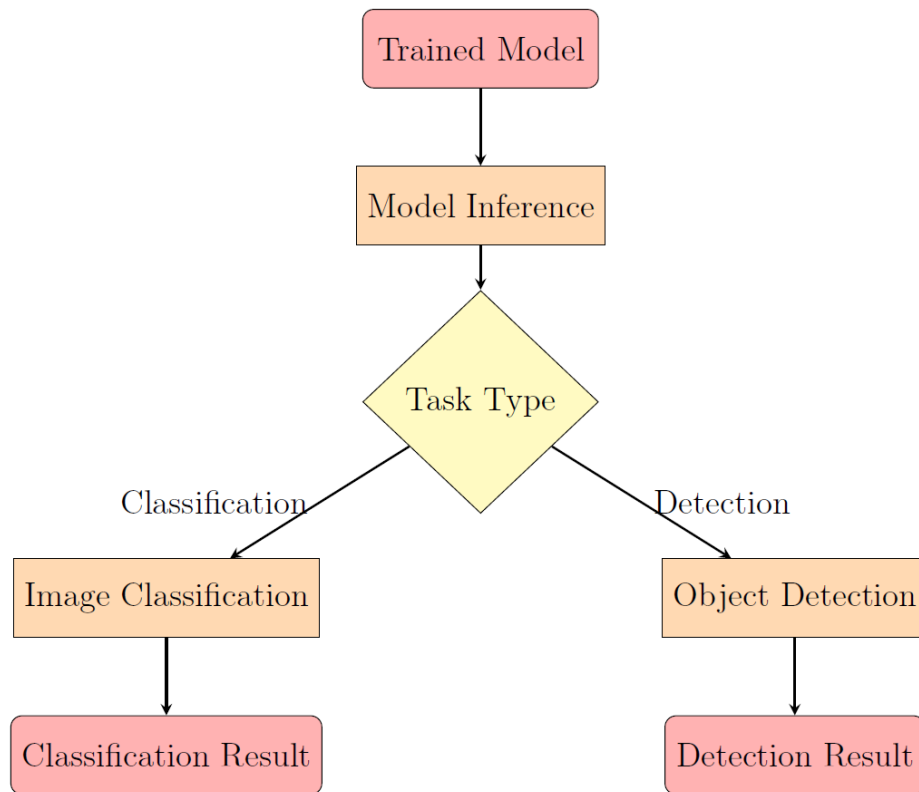
Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process



- Model Robustness evaluation

Various normal distributions corresponding to the 2nd component of the multinormal distribution during the optimization process





- Image classification in a single area of the PCB
 - Input: single PCB portion
 - Output: vector of likelihoods for each class (components)

- Image classification in a single area of the PCB
 - Input: single PCB portion
 - Output: vector of likelihoods for each class (components)

Object detection

- Component recognition on a complete PCB
 - Input: complete PCB image
 - Sliding windows approach:
 - Windows preprocessing
 - Windows classification
 - Non-Maximum Suppression with IoU criterion
 - Centroid calculation
 - Output: bounding boxes, class label, confidence score, centroid for each detected object

Object detection

Adaptive Sliding Windows Sizes

- Image Pyramid approach

- allows resizing the image to various scales and applying the sliding windows to each scale

- Detection of objects of different sizes without having to predefine the window size

- Multiple scaled versions of the same input image

Object detection

Adaptive Sliding Windows Sizes

- Image Pyramid approach
 - allows resizing the image to various scales and applying the sliding windows to each scale
 - Detection of objects of different sizes without having to predefine the window size
 - Multiple scaled versions of the same input image

Image Pyramid Algorithm

Require: Image I , scale factor $s > 1$, minimum size $(w_{\min}, h_{\min})$

Ensure: List of scaled images P

- 1: Initialize the pyramid $P \leftarrow \{I\}$
 - 2: $W \leftarrow \text{width}(I)$
 - 3: $H \leftarrow \text{height}(I)$
 - 4: **while** $W \geq w_{\min}$ **and** $H \geq h_{\min}$ **do**
 - 5: $W \leftarrow \lfloor W/s \rfloor$
 - 6: $H \leftarrow \lfloor H/s \rfloor$
 - 7: Resize the image I to (W, H)
 - 8: Add the resized image to the pyramid $P \leftarrow P \cup \{I\}$
 - 9: **end while**
 - 10: **return** P
-

Performance Optimization in Industrial Systems

Model deployment for inference in real-world industrial environments

- Beyond model **accuracy**, other key factors play a crucial role:

Performance Optimization in Industrial Systems – Key factors

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model **latency**

Performance Optimization in Industrial Systems – Key factors

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage

Performance Optimization in Industrial Systems – Key factors

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage
 - model size

Performance Optimization in Industrial Systems – Key factors

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage
 - model size
 - energy consumption

Performance Optimization in Industrial Systems – Main approaches

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage
 - model size
 - energy consumption

- To [optimize such parameters](#), two major families of approaches:

Performance Optimization in Industrial Systems – Main approaches

Model deployment for inference in real-world industrial environments

- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage
 - model size
 - energy consumption

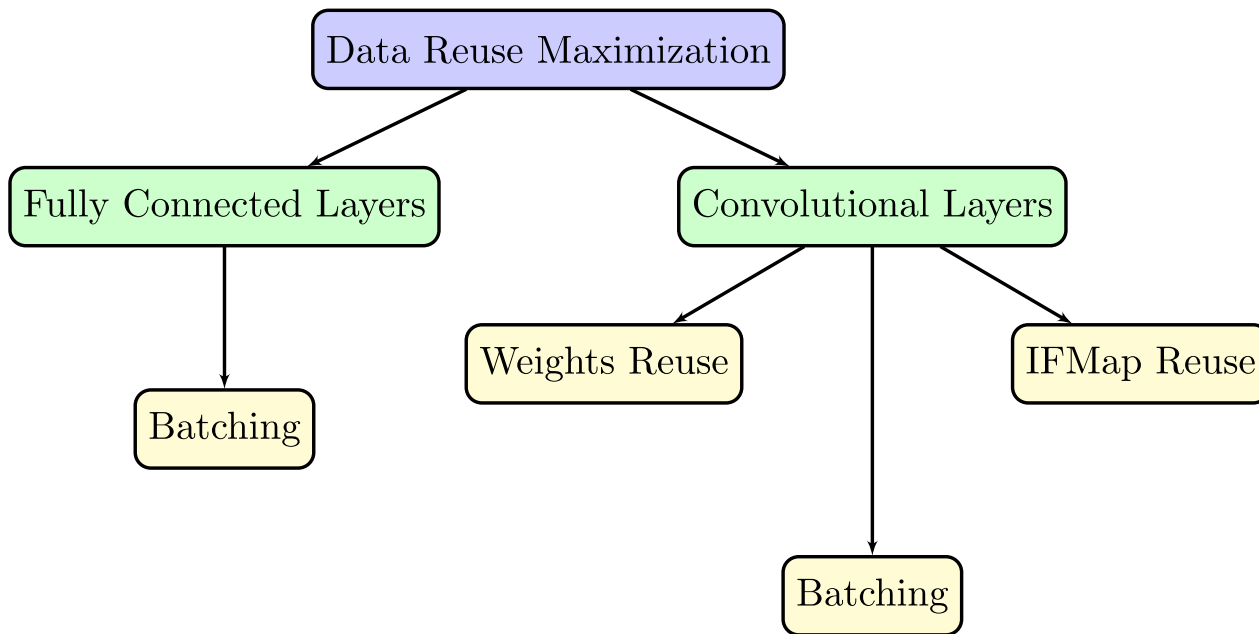
- To optimize such parameters, two major families of approaches:
 - Maximize data reuse across multiple memory hierarchy levels

Performance Optimization in Industrial Systems – Main approaches

Model deployment for inference in real-world industrial environments

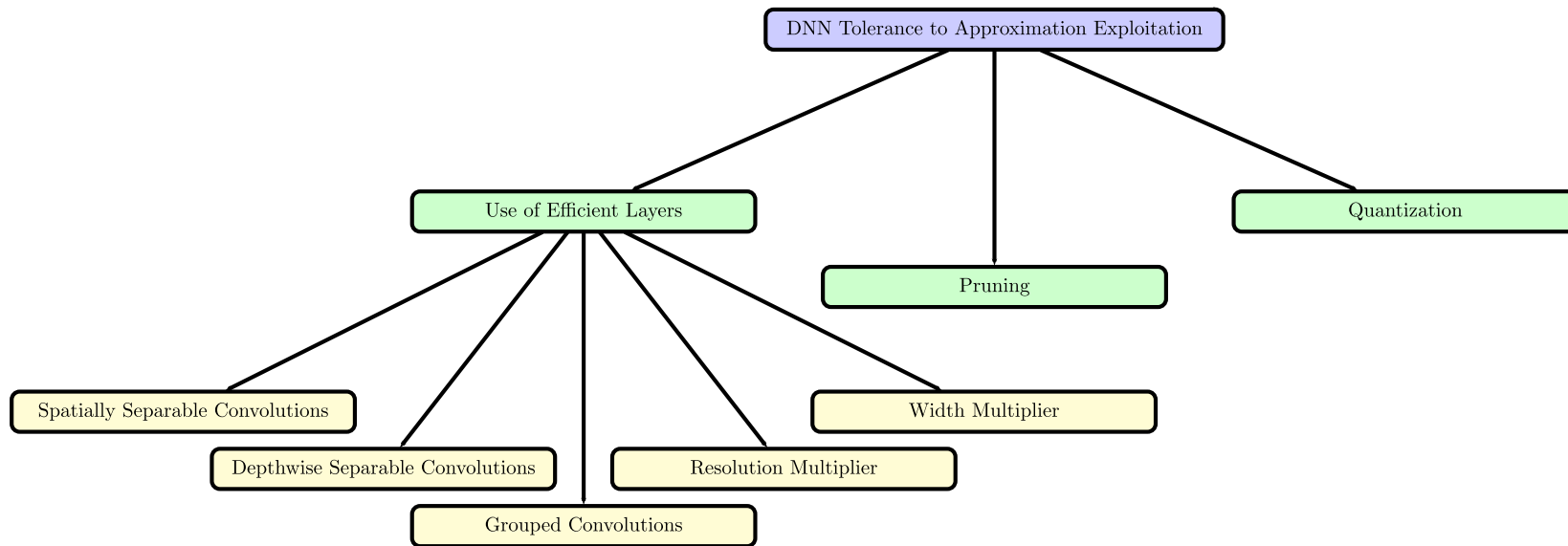
- Beyond model accuracy, other key factors play a crucial role:
 - model latency
 - memory usage
 - model size
 - energy consumption

- To optimize such parameters, two major families of approaches:
 - Maximize data reuse across multiple memory hierarchy levels
 - Exploit deep neural networks tolerance to approximation



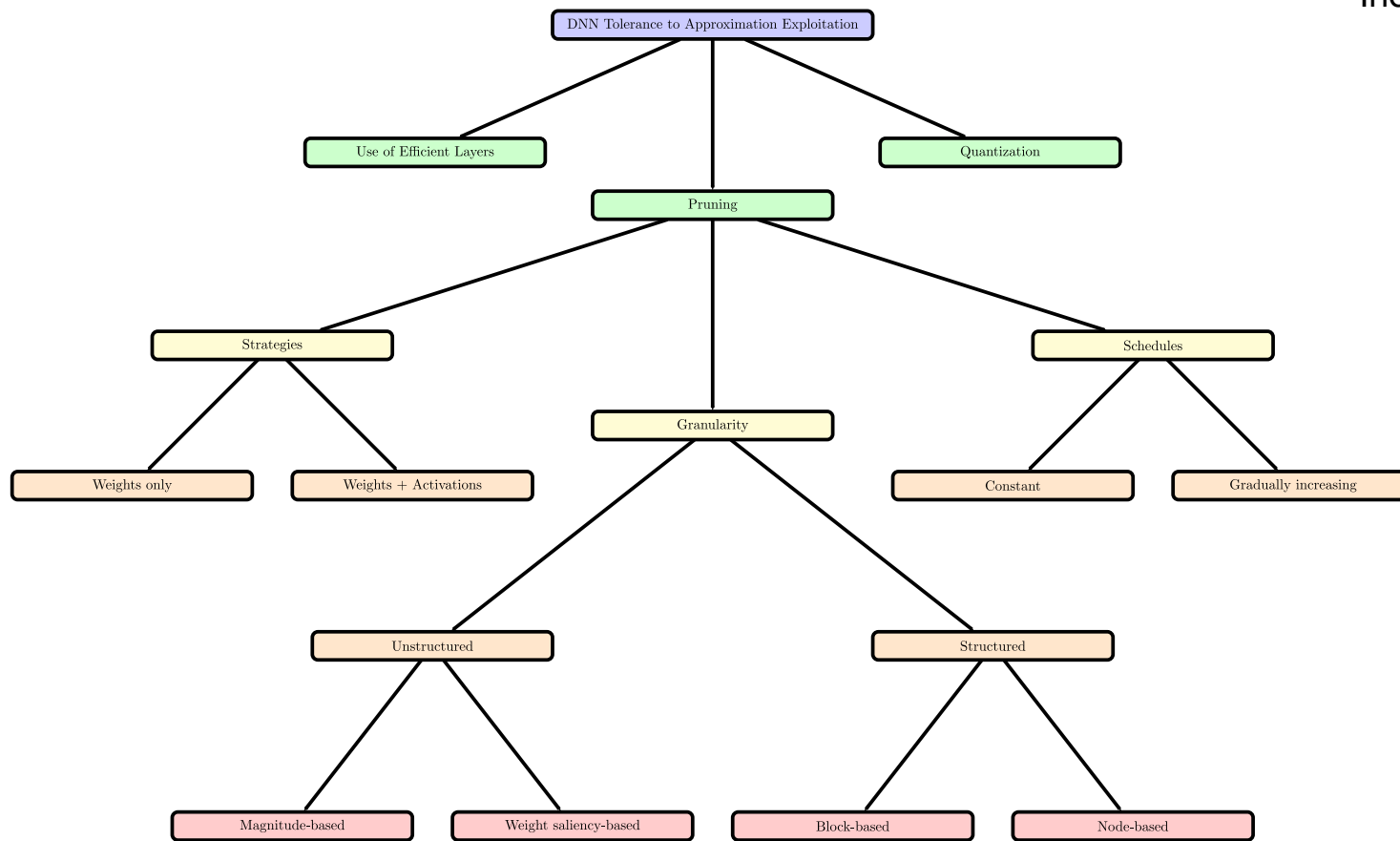
Deep Neural Network Tolerance to Approximation Exploitation

Performance
Optimization in
Industrial Systems



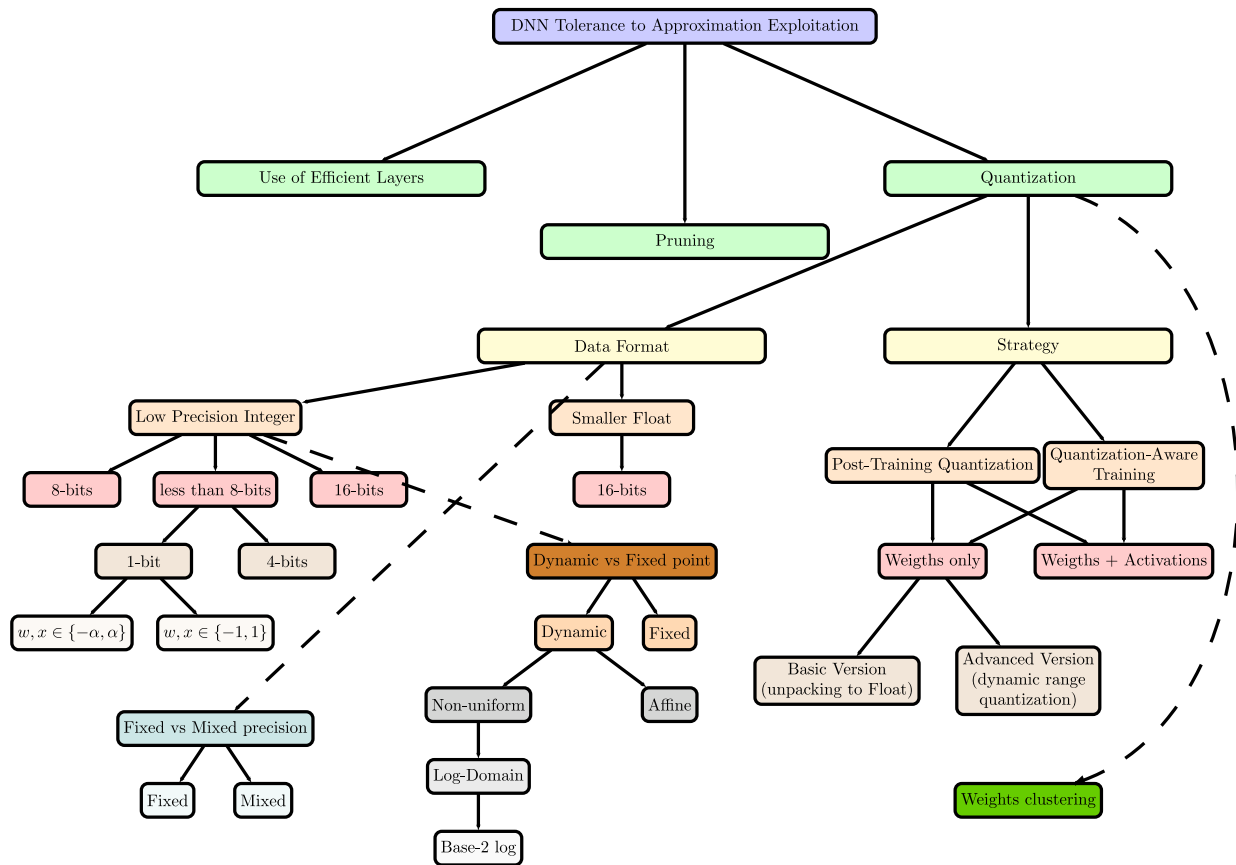
Deep Neural Network Tolerance to Approximation Exploitation

Performance
Optimization in
Industrial Systems



Deep Neural Network Tolerance to Approximation Exploitation

Performance
Optimization in
Industrial Systems



Optimization techniques effectiveness Evaluation

- Applied to real-world hardware environments
 - i.e., the hardware which manages the PCB testing machines (specifically, the Flying Probe system)

Optimization techniques effectiveness Evaluation

- Applied to real-world hardware environments
i.e., the hardware which manages the PCB testing machines
(specifically, the Flying Probe system)

with reference to Model 10, 30 different optimized versions have been evaluated
(see [Appendix](#) for details on the optimized versions)

Optimization techniques effectiveness Evaluation

- Applied to real-world hardware environments
i.e., the hardware which manages the PCB testing machines
(specifically, the Flying Probe system)

with reference to Model 10, 30 different optimized versions have been evaluated
(see Appendix for details on the optimized versions)

Models evaluated by performing 1,000 inferences, each on a different image

simulating the number of potential windows in an image representing an entire PCB

Optimization techniques effectiveness Evaluation

Performance Optimization in Industrial Systems

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Optimization techniques effectiveness Evaluation

Performance Optimization in Industrial Systems

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Binary NN



Optimization techniques effectiveness Evaluation

Performance Optimization in Industrial Systems

**Magnitude –
based pruning**

**Weight Saliency -
based pruning**

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Optimization techniques effectiveness Evaluation

Performance Optimization in Industrial Systems

16-bits QAT

16-bits QAT,
base-2 log

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Optimization techniques effectiveness Evaluation

Performance Optimization in Industrial Systems

Grouped
Convolution
(G = 32)

Grouped
Convolution
(G = 64)

Model	Accuracy (%)	Latency (s)	Memory (MB)	Energy (Wh)
Reference Model	99.98	587	763	21.32
Optimized Model 1	99.06	585	597	20.45
Optimized Model 2	95.54	491	567	17.19
Optimized Model 3	99.36	515	608	18.05
Optimized Model 4	94.93	482	562	16.83
Optimized Model 5	94.48	524	612	18.36
Optimized Model 6	94.80	559	590	19.60
Optimized Model 7	96.47	450	479	15.76
Optimized Model 8	99.87	463	501	16.23
Optimized Model 9	92.77	595	469	20.84
Optimized Model 10	94.96	514	599	18.05
Optimized Model 11	97.72	549	578	19.25
Optimized Model 12	97.57	456	487	16.07
Optimized Model 13	92.55	474	537	16.66
Optimized Model 14	94.79	511	602	17.91
Optimized Model 15	95.62	363	262	12.79
Optimized Model 16	94.56	460	490	16.16
Optimized Model 17	94.80	509	588	17.88
Optimized Model 18	95.56	430	469	15.16
Optimized Model 19	94.72	475	548	16.64
Optimized Model 20	99.67	444	488	15.69
Optimized Model 21	99.91	549	581	19.21
Optimized Model 22	99.87	462	502	16.21
Optimized Model 23	95.46	474	541	16.64
Optimized Model 24	94.31	522	607	18.25
Optimized Model 25	95.62	451	477	15.80
Optimized Model 26	98.93	510	589	17.87
Optimized Model 27	95.62	362	258	12.75
Optimized Model 28	94.95	522	611	18.26
Optimized Model 29	92.27	475	548	16.63
Optimized Model 30	91.57	592	472	20.77

Thanks for your
attention!

Appendix

A. General Models trained

Model	Feature Engineering	Model Type	Configuration Details
Model 1	Histogram of Oriented Gradients (HOG)	SVM	Kernel: Linear Regularization parameter (C): 1.0 Kernel parameter (gamma): Not applicable (used linear kernel)
Model 2	Histogram of Oriented Gradients (HOG)	SVM	Kernel: Radial Basis Function (RBF) Regularization parameter (C): 10.0 Kernel parameter (gamma): 0.01
Model 3	Principal Component Analysis (PCA)	SVM	Kernel: Polynomial Regularization parameter (C): 1.0 Degree: 3 Coefficient (coef0): 1.0
Model 4	3D Tensor	CNN	Conv1: 32 filters, kernel size 3x3, ReLU activation Conv2: 64 filters, kernel size 3x3, ReLU activation Pooling Layers: MaxPooling, pool size 2x2, MaxPooling, pool size 2x2 Fully Connected Layers: 128 units, ReLU activation, Softmax output
Model 5	3D Tensor	CNN	Conv1: 64 filters, kernel size 3x3, ReLU activation Conv2: 128 filters, kernel size 3x3, ReLU activation Conv3: 128 filters, kernel size 3x3, ReLU activation Pooling Layers: MaxPooling, pool size 2x2, MaxPooling, pool size 2x2 Fully Connected Layers: 256 units, ReLU activation, Softmax output
Model 6	3D Tensor	CNN	Conv1: 32 filters, kernel size 3x3, ReLU activation Conv2: 64 filters, kernel size 3x3, ReLU activation Conv3: 128 filters, kernel size 3x3, ReLU activation Conv4: 256 filters, kernel size 3x3, ReLU activation Pooling Layers: MaxPooling, pool size 2x2, MaxPooling, pool size 2x2, MaxPooling, pool size 2x2 Fully Connected Layers: 512 units, ReLU activation, Softmax output

Model	Feature Engineering	Model Type	Configuration Details
Model 7	3D Tensor	ResNet18 architecture	Standard ResNet18 architecture
Model 8	3D Tensor	ResNet152 architecture	Base: ResNet152 with pretrained weights on ImageNet Dense Layers: 1000 units, ReLU activation, Softmax output
Model 9	3D Tensor	ResNet152 architecture (frozen)	Base: ResNet152 with pretrained weights on ImageNet Dense Layers: 1000 units, ReLU activation, Softmax output
Model 10	3D Tensor	ResNet152 architecture	Base: ResNet152 with pretrained weights on ImageNet Dense Layers: 1024 units, ReLU activation, 2048 units, ReLU activation, 1024 units, ReLU activation, Softmax output
Model 11	3D Tensor	ResNet152 architecture (frozen)	Base: ResNet152 with pretrained weights on ImageNet Dense Layers: 1024 units, ReLU activation, 2048 units, ReLU activation, 1024 units, ReLU activation, Softmax output

B. Optimal Distribution Found

Optimal multivariate normal distribution

$$\mathcal{N}_7 \left(\begin{bmatrix} -0.117 \\ 0.407 \\ 0.984 \\ 1.227 \\ 0.810 \\ 49.788 \\ 0.369 \end{bmatrix}, \begin{bmatrix} 0.4 & 0.037 & -0.140 & -0.065 & -0.129 & -0.072 & 0.028 \\ 0.037 & 9.970 & 0.126 & -0.097 & -0.069 & 0.164 & 0.048 \\ -0.140 & 0.126 & 10.0 & 0.334 & -0.028 & -0.045 & 0.036 \\ -0.065 & -0.097 & 0.334 & 10.0 & 0.382 & 0.012 & 0.054 \\ -0.129 & -0.069 & -0.028 & 0.382 & 0.253 & 0.443 & 0.044 \\ -0.072 & 0.164 & -0.045 & 0.012 & 0.443 & 25.0 & 0.208 \\ 0.028 & 0.048 & 0.036 & 0.054 & 0.044 & 0.208 & 10.0 \end{bmatrix} \right)$$

C. Performance Optimized Models

Version	Batch size	Convolutional Layers - Data Reuse Pattern	Optimizations 1	Optimizations 2
Optimized version 1	200	Weights Reuse	Spatially Separable Convolutions, Width Multiplier ($\alpha = 0.8$)	Constant Pruning (20%), Magnitude-Based Pruning
Optimized version 2	300	IFmap Reuse	Depthwise Separable Convolutions Resolution Multiplier ($\rho = 0.75$),	Constant Pruning (40%), Block-Based Pruning (4x4)
Optimized version 3	200	Weights Reuse	Grouped Convolutions ($G = 32$)	Gradually Increasing Pruning (10%-60%), Weight Saliency-Based Pruning
Optimized version 4	300	IFmap Reuse	Spatially Separable Convolutions	Node-Based Pruning with L1 regularization ($\lambda = 0.001$), Constant Pruning (80%)
Optimized version 5	200	Weights Reuse	Depthwise Separable Convolutions, Width Multiplier ($\alpha = 0.8$)	Weights clustering (clusters = 16)
Optimized version 6	300	IFmap Reuse	Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$)	Weights clustering (clusters = 32)

Version	Batch size	Convolutional Layers - Data Reuse Pattern	Optimizations 1	Optimizations 2
Optimized version 7	200	Weights Reuse	Depthwise Separable Convolutions, Resolution Multiplier ($\rho = 0.75$)	Quantization: 8-bit integer, Base-2 log Quantization, Post-Training Quantization, Weights only (advanced version)
Optimized version 8	300	IFmap Reuse	Grouped Convolutions ($G = 32$), Width Multiplier ($\alpha = 0.8$)	Quantization: 16-bit minifloat, Quantization-Aware Training, Weights + Activation Quantization
Optimized version 9	200	Weights Reuse	Spatially Separable Convolutions	Quantization: 4-bit integer, Base-2 log Quantization, Post-Training Quantization, Weights only (advanced version)
Optimized version 10	300	IFmap Reuse	Depthwise Separable Convolutions	Weights clustering (clusters = 32)
Optimized version 11	200	Weights Reuse	Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$)	Constant Pruning (40%), Magnitude-Based Pruning
Optimized version 12	300	IFmap Reuse	Spatially Separable Convolutions, Width Multiplier ($\alpha = 0.8$)	Quantization: 8-bit integer, Dynamic fixed point Quantization, Quantization-Aware Training, Weights + Activation Quantization

Version	Batch size	Convolutional Layers - Data Reuse Pattern	Optimizations 1	Optimizations 2
Optimized version 13	200	Weights Reuse	Depthwise Separable Convolutions, Resolution Multiplier ($\rho = 0.75$)	Constant Pruning (80%), Node-Based Pruning with L1 regularization ($\lambda = 0.001$)
Optimized version 14	300	IFmap Reuse	Grouped Convolutions ($G = 32$)	Weights clustering (clusters = 16)
Optimized version 15	200	Weights Reuse	Spatially Separable Convolutions	Quantization: Binary NNs ($w, x \in \{-\alpha, \alpha\}$), Quantization-Aware Training, Weights + Activation Quantization
Optimized version 16	300	IFmap Reuse	Depthwise Separable Convolutions, Width Multiplier ($\alpha = 0.8$)	Gradually Increasing Pruning (10%-60%), Block-Based Pruning (4x4)
Optimized version 17	200	Weights Reuse	Grouped Convolutions ($G = 32$), Resolution Multiplier ($\rho = 0.75$)	Weights clustering (clusters = 16)
Optimized version 18	300	IFmap Reuse	Depthwise Separable Convolutions	Quantization: 16-bit integer, Base-2 log Quantization, Post-Training Quantization, Weights only (advanced version)

Version	Batch size	Convolutional Layers - Data Reuse Pattern	Optimizations 1	Optimizations 2
Optimized version 19	200	Weights Reuse	Grouped Convolutions ($G = 32$), Width Multiplier ($\alpha = 0.8$)	Constant Pruning (60%), Node-Based Pruning with L1 regularization ($\lambda = 0.001$)
Optimized version 20	300	IFmap Reuse	Spatially Separable Convolutions, Width Multiplier ($\alpha = 0.8$)	Quantization: 16-bit integer, Base-2 log Quantization, Quantization-Aware Training, Weights + Activation Quantization
Optimized version 21	300	Weights Reuse	Grouped Convolutions ($G = 64$), Resolution Multiplier ($p = 0.5$)	Constant Pruning (20%), Weight Saliency-Based Pruning
Optimized version 22	200	IFmap Reuse	Spatially Separable Convolutions, Width Multiplier ($\alpha = 0.95$)	Quantization: 16-bit minifloat, Quantization-Aware Training, Weights only (advanced version)
Optimized version 23	300	Weights Reuse	Depthwise Separable Convolutions	Gradually Increasing Pruning (10%-70%), Block-Based Pruning (4x4)
Optimized version 24	200	IFmap Reuse	Grouped Convolutions ($G = 32$)	Weights clustering (clusters = 64)

Version	Batch size	Convolutional Layers - Data Reuse Pattern	Optimizations 1	Optimizations 2
Optimized version 25	300	Weights Reuse	Spatially Separable Convolutions	Quantization: 8-bit integer, Base-2 log Quantization, Post-Training Quantization, Weights only (advanced version)
Optimized version 26	200	IFmap Reuse	Grouped Convolutions ($G = 64$), Width Multiplier ($\alpha = 0.7$)	Constant Pruning (40%), Magnitude-Based Pruning
Optimized version 27	300	Weights Reuse	Depthwise Separable Convolutions	Quantization: Binary NNs ($w, x \in \{-\alpha, \alpha\}$), Quantization-Aware Training, Weights + Activation Quantization
Optimized version 28	200	IFmap Reuse	Grouped Convolutions ($G = 64$)	Weights clustering (clusters = 32)
Optimized version 29	300	Weights Reuse	Spatially Separable Convolutions	Node-Based Pruning with L1 regularization ($\lambda = 0.005$), Gradually Increasing Pruning (10%-80%)
Optimized version 30	200	IFmap Reuse	Depthwise Separable Convolutions	Quantization: 4-bit integer, Dynamic fixed point Quantization, Post-Training Quantization, Weights + Activation Quantization